ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



INTRODUCTION TO ARTIFICIAL INTELLIGENCE (CO3061)

BÁO CÁO BÀI TẬP LỚN 1

TÌM KIẾM (SEARCHING)

Pipes và Light Up (Akari)

Giảng viên: Ths. Vương Bá Thịnh

Sinh viên thực hiện:

Hà Bảo Nhi	2312496	(L01)
Lê Chí Đại	2310621	(L01)
Đào Quang Dương	2310579	(L01)
Lương Minh Thuận	2313348	(L02)
Nguyễn Quốc Huy	2311209	(L02)

THÀNH PHỐ HỒ CHÍ MINH, Tháng 10 năm 2026

Mục lục

1 Pipes **2**
1.1 Giới thiệu . . . 2
1.2 Phát biểu bài toán tìm kiếm . . . 2
1.3 Tạo dữ liệu đầu vào . . . 3
1.4 Giải thuật tìm kiếm mù . . . 3
1.5 Giải thuật tìm kiếm heuristic . . . 4
1.6 Hiện thực và minh họa từng bước . . . 5
1.7 Kết quả thực nghiệm . . . 5
1.8 Nhận xét . . . 6

2 Light Up (Akari) **7**
2.1 Giới thiệu . . . 7
2.2 Phát biểu bài toán tìm kiếm . . . 7
2.3 Tạo dữ liệu đầu vào . . . 8
2.4 Giải thuật tìm kiếm mù . . . 9
2.5 Giải thuật tìm kiếm heuristic . . . 9
2.6 Hiện thực và minh họa từng bước . . . 12
2.7 Kết quả thực nghiệm . . . 12
2.8 Nhận xét . . . 13

Kết luận **14**

Phụ lục: Kết quả theo từng input Light Up **15**

Tài liệu tham khảo **17**

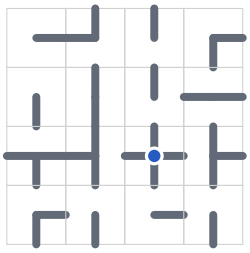
1 Pipes

1.1 Giới thiệu

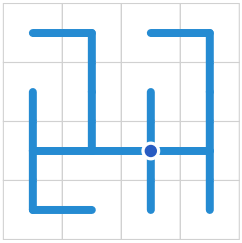
Pipes là trò chơi xoay ống trên lưới vuông $m \times m$. Nhóm lấy luật từ trang Puzzle Pipes [7]:

- Mỗi ô có một đoạn ống với hình dạng cố định: đầu cụt, ống thẳng, khuỷu, chữ T hoặc chữ thập. Người chơi chỉ được xoay ô, không đổi được hình dạng.
- Mỗi hình dạng có một số cổng mở về các phía Bắc, Đông, Nam, Tây. Xoay ô chỉ đổi hướng của các cổng, không đổi số cổng.
- Bàn được giải khi mọi cổng đều nối với cổng của ô kề, không cổng nào chìa ra ngoài biên, tất cả các ô tạo thành một mạng liên thông và mạng đó không có chu trình.

Trong chương trình, ô nguồn chỉ dùng để tô màu nước chảy qua mạng. Đổi vị trí nguồn không làm thay đổi bàn nào là hợp lệ. Hai giải thuật chỉ cần tìm một cấu hình đúng luật, không tìm cách xoay ít lần nhất.



(a) Bàn đầu vào đã xáo hướng.



(b) Lời giải do DFS tìm được.

Hình 1: Bản đồ `pipes-4x4-261.json`, vẽ từ dữ liệu thật của chương trình.

1.2 Phát biểu bài toán tìm kiếm

Gọi $n = m^2$ là số ô và đánh số các ô theo thứ tự hàng rồi cột. Ô i có tập hướng R_i gồm các cách xoay cho ra hình khác nhau. Nhóm mô hình hóa bài toán như sau:

Thành phần	Định nghĩa
Input	Bàn đã xáo hướng: kích thước, hình dạng từng ô, vị trí nguồn
Trạng thái	Dãy $s = (a_0, \ldots, a_{k-1})$: hướng đã chọn cho k ô đầu tiên
Trạng thái đầu	Dãy rỗng $()$
Hành động	Chọn một hướng trong R_k cho ô thứ k
Chuyển trạng thái	$s' = s + (a_k)$, chỉ giữ lại nếu qua được các bước kiểm tra
Trạng thái đích	$k = n$ và cả bàn thỏa luật
Chi phí	Mỗi lần gán tốn 1; mọi lời giải đều có chi phí n

Bảng 1: Mô hình tìm kiếm của Pipes.

Cần phân biệt một bước của giải thuật với một bước của người chơi. Người chơi mỗi lần xoay 90 độ, còn giải thuật chọn thẳng hướng cuối cùng cho một ô, tương ứng 0 đến 3 lần xoay. Hai giải thuật dùng chung mô hình này nên có thể so sánh trực tiếp. Bàn xáo trộn trong file chỉ

cho biết hình dạng ống; việc tìm kiếm luôn bắt đầu từ dãy rỗng và không đọc lời giải lúc sinh bàn.

Không gian tìm kiếm. Ống thẳng có 2 hướng khác nhau, chữ thập có 1, các hình còn lại có 4. Số cách gán đầy đủ là $\prod_i |R_i| \le 4^n$. Với bàn 4×4 cận trên là $4^{16} \approx 4{,}29 \times 10^9$, với bàn 7×7 là $4^{49} \approx 3{,}17 \times 10^{29}$. Riêng bản đồ ở hình trên, tích này là $268\,435\,456$. Vì các ô được gán theo thứ tự cố định nên mỗi trạng thái chỉ có một cha; cây tìm kiếm không có trạng thái lặp và không cần danh sách đã thăm.

Cắt tỉa. Trước khi tìm, chương trình bỏ các hướng làm cổng chìa ra ngoài biên. Sau đó nó kiểm tra tổng số cổng: một cây n đỉnh có $n-1$ cạnh, tức $2(n-1)$ cổng, nên nếu tổng khác số này thì bàn chắc chắn vô nghiệm. Trong lúc tìm, mỗi trạng thái con đi qua hàm `compatible()` với ba bước kiểm tra:

1. **Khớp ô đã gán.** Miền hướng của các ô chưa gán bị thu lại, chỉ giữ các hướng khớp với ô kề đã gán, kể cả phía có cổng lẫn phía không có cổng. Nếu một miền rỗng thì bỏ nhánh.
2. **Không tạo chu trình.** Các cạnh đã nối được gom bằng disjoint-set; nếu một cạnh mới nối hai ô đã cùng nhóm thì bỏ nhánh.
3. **Còn khả năng liên thông.** Chương trình dựng đồ thị gồm những cặp ô kề còn có thể mở cổng về phía nhau. Nếu đồ thị này đã bị tách rời thì không thể nối hết bàn, nên bỏ nhánh.

Ba bước này chỉ bỏ những nhánh chắc chắn không dẫn tới lời giải, nên không làm mất nghiệm. Cả DFS và Greedy dùng chung chúng; điểm khác nhau giữa hai giải thuật nằm ở thứ tự chọn trạng thái để mở rộng.

1.3 Tạo dữ liệu đầu vào

Bộ sinh bắt đầu từ ô nguồn và nối dần sang các ô chưa thăm cho tới khi phủ hết lưới, tạo thành một cây. Mỗi ô nhận hình dạng ống theo các cạnh của cây đó, rồi được xoay một góc ngẫu nhiên. Cách làm này bảo đảm bàn luôn có ít nhất một lời giải. Solver chỉ nhận bàn đã xoay, không nhận cây ban đầu.

Bộ thử gồm 9 bản đồ: ba kích thước 4×4, 5×5, 7×7, mỗi kích thước sinh bằng ba seed 261, 262, 263. Mỗi bản đồ là một file JSON lưu `size`, `source` và ma trận `tiles`, trong đó mỗi ô ghi các cổng bằng chữ N, E, S, W. Tên file dùng làm mã bản đồ.

Ngoài ra nhóm vẽ tay một bàn vô nghiệm `maps/custom/cdai.json` để kiểm tra chương trình báo vô nghiệm ngay ở gốc. Bàn này không nằm trong bảng đo.

1.4 Giải thuật tìm kiếm mù

Depth-First Search (DFS)

DFS đi sâu theo nhánh khả thi đầu tiên. Khi gặp bế tắc, nó quay lui và thử hướng kế tiếp của ô trước đó. Các hướng được thử theo thứ tự lưu sẵn trong file, không có đánh giá nào để chọn nhánh tốt hơn.

```
SEARCH_DFS(input):
  build boundary-valid domains R_i
  if root check fails: return UNSOLVABLE
  return VISIT(())

VISIT(s):
  if budget exhausted: return LIMIT_REACHED
  count one expansion
  if GOAL(s): return SOLVED(s)
  for a in boundary-valid rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):           # 3 pruning checks
      r = VISIT(child)
      if r is SOLVED or LIMIT_REACHED: return r
  return UNSOLVABLE
```

Không gian tìm kiếm hữu hạn và cắt tỉa không làm mất nghiệm, nên DFS luôn tìm được lời giải nếu bàn có nghiệm và đủ thời gian. Trường hợp xấu nhất DFS phải duyệt số trạng thái tăng theo hàm mũ của n. Về bộ nhớ, DFS chỉ giữ nhánh đang đi; mỗi mức lưu một bản sao dãy hướng, nên tổng cộng khoảng $O(n^2)$.

1.5 Giải thuật tìm kiếm heuristic

Greedy Best-First Search

Greedy giữ các trạng thái chờ trong một hàng đợi ưu tiên (min-heap) và luôn lấy ra trạng thái có h nhỏ nhất:

$$h(s) = \sum_{i \text{ chưa gán}} \log_2 |D_i(s)|,$$

trong đó $D_i(s)$ là miền hướng còn lại của ô i sau khi đã khớp với các ô kề. h chính là logarit của số cách gán còn lại cho phần chưa gán. Ví dụ ba ô còn lại có miền cỡ $(1, 2, 4)$ thì $h = 0+1+2 = 3$; nếu là $(1, 1, 2)$ thì $h = 1$, trạng thái thứ hai ít lựa chọn hơn nên được xét trước. Khi hai trạng thái có cùng h, trạng thái sâu hơn được ưu tiên, rồi tới trạng thái được đưa vào trước.

Nhóm chọn h này vì hai lý do. Thứ nhất, mọi lời giải đều cần đúng n lần gán, nên khoảng cách tới đích theo số bước không phân biệt được các trạng thái. Thứ hai, miền D_i đã được tính sẵn trong bước cắt tỉa nên tính h gần như không tốn thêm. Greedy chỉ dùng h, không cộng chi phí đã đi g như A*.

```
SEARCH_GREEDY(input):
  build boundary-valid domains; root check
  push () into min-heap with priority 0
  while heap is not empty:
    if budget exhausted: return LIMIT_REACHED
    s = pop state with lowest priority;  count one expansion
    if GOAL(s): return SOLVED(s)
    for a in boundary-valid rotations of tile |s|:
      child = s + (a);  D = FEASIBLE(child)
      if D exists: push child with (h(child), -|child|, serial)
  return UNSOLVABLE
```

Greedy cũng tìm được lời giải nếu có, vì không trạng thái khả thi nào bị bỏ khỏi heap. Đổi lại, heap phải giữ toàn bộ biên tìm kiếm, nên ở bàn khó bộ nhớ có thể tăng nhanh hơn DFS.

1.6 Hiện thực và minh họa từng bước

Mã nguồn nằm trong package `pipes_problem`: `model.py` chứa luật, kiểm tra và bộ sinh bàn; `search.py` chứa hai giải thuật và phần đo; `app.py` là giao diện pygame; `worker.py` chạy phép đo trong một process riêng; `benchmark.py` chạy cả bộ thử.

- **Chơi tay:** click để xoay ô (giữ Shift để xoay ngược), click phải để khóa ô, có Undo và Reset. Các ống đã nối với nguồn được tô màu nước.
- **Xem giải thuật chạy:** chọn DFS hoặc Greedy, rồi dùng Run, Pause, Next và Speed. Mỗi bước hiện đúng loại sự kiện: mở rộng trạng thái, gán hướng (DFS), đưa vào heap (Greedy), loại hướng (ô viền đỏ kèm lý do) và quay lui.
- **Show solution** phát lại lời giải cuối cùng từ bàn ban đầu. Chương trình báo riêng hai trường hợp "vô nghiệm" và "hết giới hạn".

1.7 Kết quả thực nghiệm

Nhóm chạy cả 9 bản đồ với hai giải thuật, mỗi cặp 5 lần, tổng cộng 90 lượt. Máy đo chạy Linux x86_64 với Python 3.13 và pygame-ce 2.5.8. Mỗi lượt giới hạn 30 giây và 200.000 lần mở rộng. Thời gian đo bằng `perf_counter` khi chạy không giao diện. Bộ nhớ là đỉnh cấp phát của Python, đo bằng `tracemalloc` ở một lần chạy lại riêng để không làm chậm phép đo thời gian; lần chạy lại phải cho cùng kết quả và cùng số lần mở rộng. Bảng dưới lấy trung vị 5 lần chạy. Số liệu thô nằm ở `docs/benchmarks/2026-10-25-pipes.csv`.

Input	Giải thuật	Giải được	Thời gian (ms)	Bộ nhớ (KiB)
pipes-4x4-261	DFS	5/5	3.380	26.836
	Greedy	5/5	2.866	11.695
pipes-4x4-262	DFS	5/5	3.047	27.078
	Greedy	5/5	3.386	12.359
pipes-4x4-263	DFS	5/5	3.149	26.984
	Greedy	5/5	2.741	12.500
pipes-5x5-261	DFS	5/5	5.352	53.203
	Greedy	5/5	7.094	19.500
pipes-5x5-262	DFS	5/5	5.906	53.344
	Greedy	5/5	5.882	19.219
pipes-5x5-263	DFS	5/5	4.564	53.297
	Greedy	5/5	5.712	19.219
pipes-7x7-261	DFS	5/5	23.247	145.852
	Greedy	5/5	27.049	27.594
pipes-7x7-262	DFS	5/5	21.938	145.930
	Greedy	5/5	26.669	27.070
pipes-7x7-263	DFS	5/5	40.785	145.852
	Greedy	5/5	38.246	29.094

Bảng 2: Pipes: thời gian và đỉnh bộ nhớ theo từng input, trung vị 5 lần chạy, đo ngày 25/10/2026. Cả 90 lượt đều giải được.

1.8 Nhận xét

Thời gian. Thời gian tăng theo kích thước bàn: khoảng 3 ms ở 4×4, 5–7 ms ở 5×5 và 22–41 ms ở 7×7. Bàn lớn có nhiều ô phải gán hơn, và mỗi lần kiểm tra cũng phải xét nhiều ô hơn.

Hai giải thuật chạy nhanh gần bằng nhau. Ở 4×4, Greedy nhanh hơn ở 2/3 bản; ở 5×5 và 7×7, DFS nhanh hơn ở 2/3 bản. Lý do là ba bước cắt tỉa đã loại gần hết các hướng sai, nên DFS gần như đi thẳng tới lời giải. Lúc đó Greedy không còn nhiều nhánh để chọn, mà lại tốn thêm thời gian tính h và thao tác với heap. Ví dụ ở bản `pipes-7x7-263`, Greedy mở rộng 83 trạng thái còn DFS mở rộng 82; Greedy nhanh hơn một chút ở bản này nhưng không phải nhờ xét ít trạng thái hơn.

Bộ nhớ. Greedy dùng ít bộ nhớ hơn DFS khoảng 2 đến 5 lần, ngược với dự đoán lý thuyết. Nguyên nhân nằm ở cách cài đặt: DFS giữ một bản sao dãy hướng và một khung generator cho mỗi mức sâu, còn heap của Greedy trên các bàn này rất nhỏ vì hầu hết nhánh đã bị cắt. Với bàn khó hơn, heap của Greedy có thể lớn lên đáng kể.

Hạn chế. Bộ thử mới có 9 bàn tương đối dễ và chưa có bàn vô nghiệm trong bảng đo. Nhóm cũng chưa đo riêng tác dụng của từng bước cắt tỉa.

2 Light Up (Akari)

2.1 Giới thiệu

Light Up (tên gốc Akari) là trò chơi đặt bóng đèn trên lưới. Nhóm lấy luật từ Nikoli và trang Puzzle Light Up [5, 6]. Bàn gồm ô trắng và ô tường màu đen; một số tường có ghi số từ 0 đến 4. Người chơi đặt bóng vào các ô trắng sao cho:

- Mỗi bóng chiếu sáng theo hàng và cột cho tới khi gặp tường hoặc biên. Ô có bóng cũng được tính là sáng.
- Mọi ô trắng đều được chiếu sáng.
- Hai bóng không được chiếu vào nhau. Hai bóng vẫn có thể cùng hàng hoặc cùng cột nếu có tường chắn ở giữa.
- Tường ghi số k có đúng k bóng ở bốn ô kề trên, dưới, trái, phải (không tính đường chéo). Tường không ghi số chỉ có tác dụng chắn sáng.

Khác với Pipes, ở đây người chơi không xoay gì cả mà chọn ô để đặt bóng. Nhóm chọn DFS làm giải thuật mù và backtracking có MRV và lan truyền ràng buộc (gọi tắt là CSP) làm giải thuật heuristic.

2.2 Phát biểu bài toán tìm kiếm

Gọi W là số ô trắng. Mỗi ô trắng i có biến $x_i \in \{0, 1\}$, với $x_i = 1$ nghĩa là ô đó có bóng. Chương trình lưu tập bóng, tập ô sáng và tập ô còn được phép đặt bóng dưới dạng bitmask để kiểm tra nhanh.

Thành phần	Định nghĩa
Input	Vị trí ô trắng, tường và số trên tường
Trạng thái	Tập bóng đã đặt, tập ô đã sáng và tập ô còn được đặt bóng
Trạng thái đầu	Chưa có bóng nào
Hành động	DFS: đặt một bóng. CSP: đặt bóng tại một nguồn chiếu, hoặc chọn một tổ hợp bóng quanh tường
Chuyển trạng thái	Cập nhật bóng, ô sáng và ô còn đặt được; CSP suy luận thêm các bóng bắt buộc
Trạng thái đích	Mọi ô trắng sáng, không hai bóng chiếu nhau, mọi tường số đủ bóng
Chi phí	Số bóng của lời giải; cả hai giải thuật chỉ tìm lời giải đầu tiên, không tối ưu số bóng

Bảng 3: Mô hình tìm kiếm của Light Up.

Viết dưới dạng ràng buộc, gọi $V(i)$ là các ô trắng nhìn thấy từ i (kể cả i), L là một đoạn ô

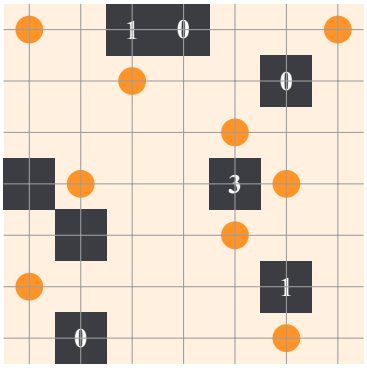
trắng liền nhau trên một hàng hoặc cột, và $N(w)$ là các ô trắng kề tường w có số k_w:

$$\sum_{j \in V(i)} x_j \geq 1 \qquad \text{(ô } i \text{ được chiếu sáng)},$$

$$\sum_{j \in L} x_j \leq 1 \qquad \text{(không hai bóng chiếu nhau)},$$

$$\sum_{j \in N(w)} x_j = k_w \qquad \text{(tường } w \text{ đủ bóng)}.$$

Hàm `is_goal_state()` kiểm tra ba điều kiện này. Số tập bóng có thể có là 2^W; đây là kích thước không gian cấu hình, còn số trạng thái thực sự được mở rộng nhỏ hơn rất nhiều nhờ cắt tỉa.



Hình 2: Bàn 7×7, mức easy, seed 70097 và lời giải do CSP tìm được. Ô đen là tường, hình tròn là bóng đèn.

2.3 Tạo dữ liệu đầu vào

Bàn được sinh theo seed, nên cùng seed luôn cho cùng một bàn. Có ba mức easy, normal và hard, khác nhau ở tỉ lệ tường ρ. Bàn $n \times n$ có $M = \text{round}(\rho n^2)$ ô tường:

Mức	ρ	7×7	10×10	14×14	25×25
easy	16%	8	16	31	100
normal	10.5%	5	10	21	66
hard	5.5%	3	6	11	34

Các bước sinh bàn (`light_up_problem/generator/generate.py`):

1. Chọn ngẫu nhiên M vị trí làm tường.
2. Lần lượt đặt bóng vào một ô trắng còn tối cho tới khi mọi ô đều sáng. Tập bóng này là một lời giải mẫu.
3. Ghi số lên tường bằng số bóng kề trong lời giải mẫu, để khoảng 25% số tường không ghi số. Cuối cùng kiểm tra lại lời giải mẫu với luật.

Vì bàn được dựng từ một lời giải nên chắc chắn có nghiệm. Solver không được biết lời giải mẫu. Nhóm có tham khảo cách đánh giá độ khó theo kỹ thuật suy luận của Dekker [2], nhưng chưa áp dụng; nhãn easy/normal/hard hiện chỉ phản ánh số tường, và chương trình chưa kiểm tra bàn có nghiệm duy nhất.

2.4 Giải thuật tìm kiếm mù

Depth-First Search (DFS)

Ở mỗi bước, DFS lấy ô tối đầu tiên theo thứ tự hàng rồi cột và thử lần lượt các vị trí có thể đặt bóng để chiếu ô đó. Một vị trí bị bỏ nếu bóng chiếu vào bóng khác, làm một tường bị thừa bóng hay không còn đủ chỗ đặt bóng, hoặc làm một ô tối không còn chỗ nào chiếu tới được. DFS đi sâu ngay vào nhánh hợp lệ đầu tiên và quay lui khi bế tắc.

```
DFS(state):
  if budget exhausted: return LIMIT
  if GOAL(state): return SOLVED(state)
  target = first dark cell in row-major order
  for bulb in legal illuminators(target):
    child = place(bulb)
    if FEASIBLE(child):
      result = DFS(child)
      if solved or limited: return result
    exclude bulb from later sibling choices
  return UNSOLVABLE
```

Dòng `exclude bulb` giúp các nhánh anh em không đặt lại một vị trí đã thử, tránh việc cùng một tập bóng xuất hiện nhiều lần với thứ tự đặt khác nhau. Trong code, đệ quy được thay bằng stack tường minh. Thứ tự chọn ô và thứ tự thử là cố định, DFS không dùng điểm đánh giá nào.

Mỗi bước thêm một bóng nên độ sâu không quá W. Nếu mỗi ô tối có tối đa b cách chiếu và lời giải sâu d, số trạng thái xấu nhất là $O(b^d)$. Stack chỉ giữ nhánh hiện tại, nhưng chương trình còn lưu liên kết cha để phát lại các bước, nên bộ nhớ tăng theo số trạng thái đã sinh.

2.5 Giải thuật tìm kiếm heuristic

Backtracking với MRV và lan truyền ràng buộc (CSP)

Ý tưởng là suy ra những gì bắt buộc trước, chỉ đoán khi không suy ra được nữa, và khi phải đoán thì đoán ở chỗ ít lựa chọn nhất. Cách làm này dựa trên phân tích của Pulles [3] và bộ giải Borroot/akari [4].

Trạng thái và tên các bước. Dùng thống nhất `state = (bulbs, lit, legal)`: tập bóng đã đặt, tập ô đã sáng và tập vị trí còn được phép đặt bóng. Gọi P là tập ô trắng, $dark = P \setminus lit$, $V(i)$ là tập ô nhìn thấy từ i (kể cả i), $N(w)$ là tập ô trắng kề tường w, và k_w là số ghi trên tường. Các tập này được lưu bằng bitmask trong code. `PROPAGATE` là bước lan truyền ràng buộc; `SELECT_MRV_CHOICES` chọn ràng buộc bằng MRV và trả các nhánh theo thứ tự gain; `GOAL` kiểm tra đủ ba điều kiện thắng.

Lan truyền ràng buộc (CP): `PROPAGATE`. Bước này lặp các luật sau tới điểm cố định (không còn thay đổi), trước khi chọn nhánh:

- Tường đã đủ bóng thì cấm đặt bóng ở các ô kề còn lại. Tường còn thiếu r bóng và chỉ còn đúng r ô kề đặt được thì đặt bóng ở cả r ô đó.
- Ô tối chỉ còn một vị trí có thể chiếu tới thì đặt bóng ở vị trí đó. Mỗi bóng mới làm cấm các ô khác trên đường sáng của nó.
- Nếu đặt bóng ở một ô làm một tường số không thể đủ bóng, ô đó bị cấm.

Nếu gặp mâu thuẫn (ô tối không còn nguồn chiếu, tường không thể đủ bóng) thì nhánh hiện tại bị bỏ.

Chọn ràng buộc bằng MRV: `SELECT_MRV_CHOICES`**.** Sau `PROPAGATE`, nếu chưa tới đích, MRV (Minimum Remaining Values) so sánh số lựa chọn của hai loại ràng buộc trên cùng trạng thái:

- **Tường số** w**:** số bóng còn thiếu là $r = k_w - |N(w) \cap bulbs|$; tập ô kề còn được phép đặt là $U(w) = N(w) \cap legal$, có $u = |U(w)|$ phần tử. Với $r > 0$, điểm là

$$m_{\text{wall}}(w) = \binom{u}{r}.$$

  Mỗi lựa chọn đặt đúng r bóng vào một tổ hợp của $U(w)$ và cấm các ô kề không được chọn. Tường đã đủ bóng ($r = 0$) không tham gia MRV; $r < 0$ hoặc $r > u$ là mâu thuẫn được phát hiện trong `PROPAGATE`.
- **Ô tối** i**:** tập nguồn chiếu còn được phép là $C(i) = V(i) \cap legal$. Điểm là

$$m_{\text{dark}}(i) = |C(i)|.$$

  Mỗi lựa chọn đặt một bóng tại một nguồn chiếu trong $C(i)$; $C(i) = \varnothing$ là mâu thuẫn. Tập $legal$ đã loại các ô bị CP cấm, không chỉ xét ô có bóng hay chưa.

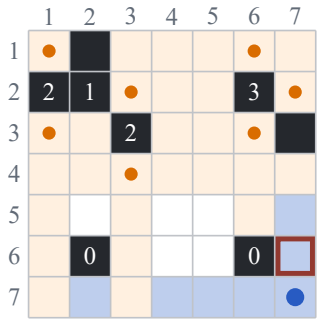
Chọn ràng buộc có điểm nhỏ nhất trong cả hai nhóm; bằng điểm thì ưu tiên tường. Trong cùng loại, giữ phần tử gặp trước theo thứ tự duyệt của code. Ví dụ, tường thiếu $r = 2$ bóng và còn $u = 3$ ô kề có điểm 3; ô tối có hai nguồn chiếu có điểm 2, nên chọn ô tối. Nếu ô tối cũng có điểm 3 thì chọn tường. Điểm $\binom{u}{r}$ được tính *trước* khi sinh và loại các tổ hợp có bóng chiếu nhau, nên không nhất thiết bằng số nhánh thực tế còn lại.

Sắp thứ tự thử bằng gain. Sau khi MRV chọn ràng buộc, gọi A là tập bóng mới của một nhánh: một bóng nếu chọn ô tối, hoặc r bóng nếu chọn tường. Điểm chiếu sáng thêm là

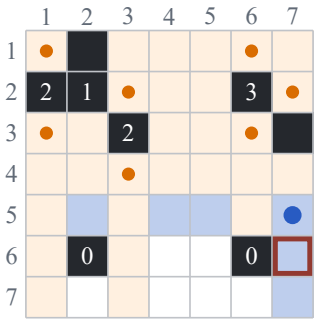
$$gain(A) = \left| \left(\bigcup_{b \in A} V(b) \right) \cap dark \right| = |lit_{\text{child}} \setminus lit|.$$

Lấy hợp trước khi đếm để một ô được nhiều bóng chiếu vẫn chỉ tính một lần. Thử nhánh có gain lớn trước; bằng gain giữ thứ tự sinh ban đầu. Gain được tính ngay sau khi đặt A, *trước CP của trạng thái con*; không cộng gain vào điểm MRV và không dùng nó để chọn giữa tường với ô tối.

Ví dụ thực từ bàn 7×7 easy, seed 70098: sau CP ở gốc, ô tối $(6, 7)$ có hai nguồn chiếu $(7, 7)$ và $(5, 7)$ (tọa độ hàng, cột tính từ 1). Gain tương ứng là 7 và 6, nên thử $(7, 7)$ trước (Hình 3).



(a) Đặt bóng tại $(7, 7)$: gain $= 7$.



(b) Đặt bóng tại $(5, 7)$: gain $= 6$.

Hình 3: Ví dụ thực từ bàn 7×7 easy, seed 70098, sau CP ở gốc và trước CP của con. Viền đỏ đánh dấu ô tối $(6, 7)$; chấm cam là bóng đã đặt, chấm xanh là bóng thử. Nền cam nhạt là ô đã sáng, nền xanh là ô mới sáng dùng để tính gain. Tọa độ hàng và cột tính từ 1.

Khi chọn nguồn chiếu, các nhánh sau cấm các nguồn đã thử ở nhánh trước: nhánh thứ hai đặt tại $(5, 7)$ đồng thời cấm $(7, 7)$. Các nhánh vì thế không lặp cùng tập bóng theo thứ tự đặt khác nhau. Với tường, tổ hợp không hợp lệ bị loại trước khi xếp gain; các mâu thuẫn còn lại được CP của con kiểm tra.

Quay lui: `BACKTRACK_MRV_CP`**.** Mã giả dùng cùng tên các bước ở trên. `SELECT_MRV_CHOICES` trả các trạng thái con đã áp dụng lựa chọn và điều kiện cấm của từng nhánh, xếp theo gain giảm dần.

```
BACKTRACK_MRV_CP(state):
  if budget exhausted: return LIMIT
  state, ok = PROPAGATE(state)
  if not ok: return UNSOLVABLE
  if GOAL(state): return SOLVED(state)
  checkpoint = state
  choices = SELECT_MRV_CHOICES(checkpoint)
  for child in choices:  # descending gain, before child CP
    result = BACKTRACK_MRV_CP(child)
    if result is SOLVED or LIMIT: return result
    state = checkpoint
  return UNSOLVABLE
```

Checkpoint giữ đủ `(bulbs, lit, legal)` sau CP của nút cha. Khi quay lui, khôi phục cả ba tập trước khi thử nhánh kế tiếp. Code dùng stack tường minh thay cho đệ quy; `PROPAGATE` tương ứng `propagate`, còn `SELECT_MRV_CHOICES` tương ứng `select_mrv_choices` trong `algorithms/search.py`. Mã giả gom các trường hợp dừng do tài nguyên thành `LIMIT`; code kiểm tra giới hạn trong quá trình duyệt, và trong CP khi bật trace.

Trường hợp xấu nhất CSP vẫn tốn thời gian hàm mũ. CP giúp giảm mạnh số nhánh nhưng làm mỗi nút tốn công hơn, vì phải lặp các luật suy luận. Một nút CSP có thể đặt nhiều bóng cùng lúc, nên số nút của CSP và DFS không tương ứng với cùng một lượng công việc. Cả hai giải thuật chỉ bỏ những nhánh chắc chắn không có nghiệm, nên sẽ tìm được lời giải nếu đủ thời gian.

2.6 Hiện thực và minh họa từng bước

Package `light_up_problem` gồm `core/` (luật và cấu trúc bàn), `generator/` (sinh bàn), `algorithms/search.py` (giải thuật), `ui/app.py` (giao diện pygame-ce và pygame-gui) và `benchmark/` (đo). Solver chạy trong một worker riêng để giao diện không bị đứng.

- **Chơi tay:** click trái để đặt hoặc bỏ bóng, click phải để đánh dấu ô không đặt bóng, rồi bấm kiểm tra lời giải.
- **Xem giải thuật chạy:** chọn DFS hoặc CSP rồi bấm Start Search. Có Pause/Resume, hai mũi tên để đi tới hoặc lui từng bước, và ba tốc độ 2, 8, 30 bước mỗi giây.
- DFS hiện ô đang xét, bóng vừa đặt và lúc quay lui. CSP hiện thêm các bóng được suy ra, các ô bị cấm (dấu $\times$) và việc khôi phục khi quay lui.
- Nút **Compare DFS and CSP** chạy cả hai giải thuật một lần trên bàn đang mở và hiện thời gian, bộ nhớ, số nút.

2.7 Kết quả thực nghiệm

Nhóm đo trên 4 kích thước, 3 mức và 5 seed cho mỗi cấu hình, tức 60 bàn khác nhau. Mỗi bàn được giải một lần bằng mỗi giải thuật, tổng cộng 120 lượt. Mỗi lượt chạy trong một process mới, các lượt chạy lần lượt và thứ tự hai giải thuật được đảo ở các seed liên tiếp. Seed dùng cho từng cấu hình:

Cỡ	easy	normal	hard
7	70097–70101	70194–70198	70291–70295
10	100097–100101	100194–100198	100291–100295
14	140097–140101	140194–140198	140291–140295
25	250097–250101	250194–250198	250291–250295

Mỗi lượt giới hạn 30 giây, 500.000 nút mở rộng và 512 MiB bộ nhớ. Lượt vượt giới hạn được ghi là TIMEOUT, NODE_LIMIT hoặc MEMORY_LIMIT, không bị tính là vô nghiệm. Thời gian đo bằng `perf_counter` quanh solver, không tính thời gian sinh bàn hay khởi động process. Bộ nhớ là đỉnh cấp phát của Python đo bằng `tracemalloc`, không phải RAM của cả process. Mọi lời giải đều được kiểm tra lại bằng `is_goal_state()`. Số liệu thô nằm ở `docs/benchmarks/2026-10-31-lightup-dfs-csp.csv`.

Giải thuật	SOLVED	TIMEOUT	NODE_LIMIT	MEMORY_LIMIT
DFS	52	8	0	0
CSP	60	0	0	0

Cỡ	Mức	Giải thuật	Giải được	Thời gian (s)	Bộ nhớ (KiB)	Số nút	Số bóng
7	easy	DFS	5/5	0.0102	27.1	30.6	8.4
7	easy	CSP	5/5	0.0078	25.1	4.0	8.4
7	normal	DFS	5/5	0.0067	26.8	9.2	7.6
7	normal	CSP	5/5	0.0103	29.1	6.2	7.2
7	hard	DFS	5/5	0.0072	27.3	8.4	7.4
7	hard	CSP	5/5	0.0105	30.5	6.4	6.2
10	easy	DFS	5/5	0.0360	58.7	51.6	15.4
10	easy	CSP	5/5	0.0280	56.0	7.8	13.8
10	normal	DFS	5/5	0.0280	62.6	40.2	13.0
10	normal	CSP	5/5	0.0312	62.2	8.8	11.6
10	hard	DFS	5/5	0.0230	62.4	12.4	11.4
10	hard	CSP	5/5	0.0314	67.7	9.8	10.0
14	easy	DFS	5/5	0.9070	228.9	983.4	26.2
14	easy	CSP	5/5	0.0947	108.3	12.4	24.4
14	normal	DFS	5/5	1.3044	375.7	2 020.2	21.6
14	normal	CSP	5/5	0.1033	123.7	16.0	20.0
14	hard	DFS	5/5	0.0858	126.2	24.0	18.6
14	hard	CSP	5/5	0.1177	134.3	14.6	15.2
25	easy	DFS	2/5	23.7947	2192.9	8 678.4	73.0
25	easy	CSP	5/5	2.1056	579.9	96.2	67.8
25	normal	DFS	1/5	24.2972	3522.0	13 406.6	58.0
25	normal	CSP	5/5	1.0444	693.0	33.8	52.6
25	hard	DFS	4/5	7.7500	1268.7	2 407.8	41.0
25	hard	CSP	5/5	1.0463	872.3	33.6	35.2

Bảng 4: Light Up: trung bình cộng (mean) trên 5 seed của mỗi cấu hình. Thời gian, đỉnh bộ nhớ Python và số nút tính cả lượt TIMEOUT: thời gian là thời lượng chạy tới lúc dừng, không phải thời gian giải xong. Số bóng chỉ lấy trung bình các lượt SOLVED, với số mẫu ở cột Giải được. Kết quả từng bàn ở phụ lục.

2.8 Nhận xét

Số bàn giải được và số nút. CSP giải được 60/60 bàn, DFS giải được 52/60. Tám lượt TIMEOUT đều của DFS, ở bàn 25×25; không lượt nào chạm giới hạn nút hay bộ nhớ. Tại 25×25 normal, DFS mở trung bình 13 406.6 nút so với 33.8 nút của CSP. Ở cỡ 25, CSP mở trung bình 33.6–96.2 nút tùy nhãn. CP có thể suy ra nhiều bóng trong một nút, nên các số này không đại diện cùng lượng công việc giữa hai giải thuật.

Thời gian. Trong 52 bàn cả hai cùng giải được, CSP nhanh hơn ở 15 bàn, DFS nhanh hơn ở 37 bàn. Đây là số đếm theo từng cặp, khác với trung bình theo cấu hình trong Bảng 4. Với bàn nhỏ, chi phí lặp các luật CP có thể khiến CSP chậm hơn DFS; tuy nhiên CSP có mean thời gian thấp hơn DFS ở 7×7 easy và 10×10 easy. Tại 14×14 easy, mean DFS/CSP là 0.9070/0.0947 giây, tỉ số khoảng 9.6; ở normal là 1.3044/0.1033 giây, tỉ số khoảng 12.6. Một số lượt DFS dài làm mean tăng rõ rệt, nên không suy rộng các tỉ số này cho mọi seed. Ở 25×25, CSP có mean 1.0444–2.1056 giây; mean thời lượng lượt chạy DFS là 7.7500–24.2972 giây, có tính các lượt bị dừng sau 30 giây. Vì có TIMEOUT, tỉ số thời lượng trung bình ở các nhóm này không phải tỉ số thời gian giải xong trên cùng tập bàn.

Bộ nhớ và số bóng. Ở cỡ 7 và 10, mean đỉnh bộ nhớ Python của hai giải thuật khá gần nhau. Tại 25×25 easy và normal, mean DFS/CSP lần lượt là 2192.9/579.9 KiB và 3522.0/693.0 KiB, tức khoảng 3.8 và 5.1 lần; ở hard tỉ số khoảng 1.5 lần. DFS phải giữ nhiều trạng thái đã sinh để

phát lại đường đi. Cột Số bóng là mean riêng trên các lượt SOLVED: khi số lượt giải được khác nhau, hai mean có thể dựa trên tập seed khác nhau. Trong 52 cặp cùng giải được, CSP dùng ít bóng hơn ở 39 bàn; cả hai không bảo đảm tối ưu số bóng.

Hạn chế. Mỗi cấu hình mới có 5 seed; mean nhạy với các lượt chạy dài và các lượt bị giới hạn. Nhãn độ khó chỉ dựa trên số tường; ở cỡ 14 và 25, mean thời lượng DFS ở hard thấp hơn easy và normal. Chương trình chưa kiểm tra tính duy nhất của lời giải và chưa đo riêng tác dụng của MRV, gain và CP. Các lượt đo cũ ngày 25/10 dùng generator trước đó được lưu riêng, không trộn vào bảng thực nghiệm mới.

Kết luận

Nhóm đã hiện thực hai trò chơi Pipes và Light Up bằng Python 3. Mỗi trò chơi có chế độ chơi tay, chế độ xem giải thuật chạy từng bước, và hai giải thuật: một giải thuật mù (DFS) và một giải thuật có heuristic.

- **Pipes:** DFS và Greedy Best-First đều giải được cả 9 bàn trong vài chục mili giây. Vì cắt tỉa đã loại gần hết hướng sai, hai giải thuật chạy nhanh gần như nhau; Greedy dùng ít bộ nhớ hơn trên bộ thử này.
- **Light Up:** CSP (backtracking với MRV và lan truyền ràng buộc) giải được 60/60 bàn, DFS giải được 52/60. Ở bàn 25×25, thời gian trung bình của CSP khoảng 1.04–2.11 giây tùy nhãn; DFS có 8 lượt bị dừng sau 30 giây. Các giá trị trung bình trong bảng tính cả lượt TIMEOUT, nên không coi thời lượng lượt bị dừng là thời gian giải xong.

Bài học chính của nhóm là heuristic không phải lúc nào cũng làm giải thuật nhanh hơn: nó có ích khi không gian tìm kiếm đủ lớn, còn ở bài dễ thì chi phí tính heuristic có thể lớn hơn phần tiết kiệm được. Hướng phát triển tiếp theo là thêm bàn khó và bàn vô nghiệm, đo nhiều seed hơn, kiểm tra tính duy nhất của lời giải Light Up và đo riêng tác dụng của từng kỹ thuật cắt tỉa, MRV và CP.

Phụ lục: Kết quả theo từng input Light Up

Mỗi dòng là một bàn, giải một lần bằng mỗi giải thuật. Thời gian tính bằng giây, bộ nhớ là đỉnh cấp phát Python tính bằng KiB. Cột TT DFS ghi S (SOLVED) hoặc T (TIMEOUT); với lượt T, thời gian là lúc bị dừng. CSP giải được tất cả các bàn. Số liệu đo ngày 31/10/2026.

				DFS		CSP	
Cỡ	Nhãn	Seed	TT DFS	Giây	KiB	Giây	KiB
7	easy	70097	S	0.0075	24.9	0.0115	25.4
7	easy	70098	S	0.0084	25.4	0.0057	24.8
7	easy	70099	S	0.0236	36.7	0.0066	25.2
7	easy	70100	S	0.0057	24.8	0.0062	24.8
7	easy	70101	S	0.0060	23.8	0.0091	25.5
7	normal	70194	S	0.0066	27.1	0.0103	28.6
7	normal	70195	S	0.0063	26.7	0.0106	29.4
7	normal	70196	S	0.0070	26.8	0.0104	29.0
7	normal	70197	S	0.0063	26.6	0.0098	29.8
7	normal	70198	S	0.0071	26.9	0.0102	28.5
7	hard	70291	S	0.0079	27.2	0.0109	30.0
7	hard	70292	S	0.0079	27.3	0.0108	30.9
7	hard	70293	S	0.0064	27.6	0.0095	30.6
7	hard	70294	S	0.0065	27.3	0.0107	30.5
7	hard	70295	S	0.0074	27.1	0.0106	30.3
10	easy	100097	S	0.0189	55.1	0.0222	54.7
10	easy	100098	S	0.0229	55.9	0.0244	55.0
10	easy	100099	S	0.0203	54.9	0.0313	56.2
10	easy	100100	S	0.0495	59.1	0.0288	56.9
10	easy	100101	S	0.0685	68.2	0.0331	57.0
10	normal	100194	S	0.0201	59.0	0.0323	62.8
10	normal	100195	S	0.0204	59.0	0.0309	61.4
10	normal	100196	S	0.0230	61.2	0.0279	61.1
10	normal	100197	S	0.0218	60.0	0.0341	63.6
10	normal	100198	S	0.0548	73.5	0.0305	61.8
10	hard	100291	S	0.0238	61.6	0.0334	67.8
10	hard	100292	S	0.0237	62.4	0.0284	66.2
10	hard	100293	S	0.0229	62.4	0.0348	67.2
10	hard	100294	S	0.0223	62.3	0.0293	68.2
10	hard	100295	S	0.0223	63.3	0.0314	69.2

Bảng 5: Kết quả từng input Light Up, cỡ 7 và 10.

				DFS		CSP	
Cỡ	Nhãn	Seed	TT DFS	Giây	KiB	Giây	KiB
14	easy	140097	S	3.5735	593.5	0.1642	110.4
14	easy	140098	S	0.4546	176.2	0.0883	110.9
14	easy	140099	S	0.2330	135.3	0.0680	105.0
14	easy	140100	S	0.0701	110.1	0.0801	106.1
14	easy	140101	S	0.2036	129.3	0.0730	109.1
14	normal	140194	S	0.0596	117.3	0.0902	126.3
14	normal	140195	S	0.8866	242.5	0.0883	122.0
14	normal	140196	S	0.0647	116.6	0.1030	122.4
14	normal	140197	S	5.4386	1284.9	0.1190	123.5
14	normal	140198	S	0.0724	117.1	0.1160	124.2
14	hard	140291	S	0.0815	125.2	0.1168	135.7
14	hard	140292	S	0.0991	126.1	0.1124	134.3
14	hard	140293	S	0.0841	126.8	0.1159	134.6
14	hard	140294	S	0.0809	125.8	0.1262	135.0
14	hard	140295	S	0.0832	127.0	0.1172	132.0
25	easy	250097	S	18.3397	2440.6	6.7115	656.3
25	easy	250098	T	30.0018	2495.0	0.9558	550.6
25	easy	250099	T	30.0001	2237.0	0.9504	562.1
25	easy	250100	S	10.6316	1572.3	0.9626	554.0
25	easy	250101	T	30.0001	2219.6	0.9475	576.4
25	normal	250194	S	1.4799	797.0	1.0059	696.9
25	normal	250195	T	30.0009	3583.3	0.9933	711.0
25	normal	250196	T	30.0002	6332.1	1.0680	697.5
25	normal	250197	T	30.0004	4732.9	1.1273	681.6
25	normal	250198	T	30.0044	2164.9	1.0274	677.9
25	hard	250291	S	0.7432	828.5	1.0389	856.5
25	hard	250292	S	0.7423	850.3	1.0203	879.0
25	hard	250293	T	30.0000	1938.4	1.0464	855.0
25	hard	250294	S	0.7835	870.3	1.0421	902.7
25	hard	250295	S	6.4809	1855.9	1.0838	868.0

Bảng 6: Kết quả từng input Light Up, cỡ 14 và 25.

Tài liệu tham khảo

[1] B. McPhail. *Light Up is NP-complete*, 2005. https://www.mountainvistasoft.com/docs/lightup-is-np-complete.pdf

[2] V. Dekker. *A Difficulty Measure for Light Up Puzzles*. Luận văn cử nhân, Leiden University, 2015. https://theses.liacs.nl/pdf/2014-2015Victor-Dekker.pdf

[3] B. Pulles. *Analysis of Akari*. Luận văn cử nhân, Radboud University, 2021; mục 4.1.1 và 4.1.3. https://www.cs.ru.nl/bachelors-theses/2021/Bram_Pulles___1015194___Analysis_of_Akari.pdf

[4] Borroot. *akari*: mã nguồn bộ giải, đặc biệt `src/tracksolver.py`. https://github.com/borroot/akari

[5] Nikoli. *Akari*: luật chơi chính thức. https://www.nikoli.co.jp/en/puzzles/akari/

[6] Puzzle Light Up. Trò chơi trực tuyến và giao diện tham khảo. https://www.puzzle-light-up.com/

[7] Puzzle Pipes. https://www.puzzle-pipes.com/

[8] S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Pearson, 2021 (chương 3: Tìm kiếm trong không gian trạng thái; chương 6: Ràng buộc).

[9] Python 3 documentation: `time`, `tracemalloc`, `multiprocessing`, `heapq`, `unittest`. https://docs.python.org/3/

[10] pygame-ce documentation. https://pyga.me/docs/ pygame_gui documentation. https://pygame-gui.readthedocs.io/